

task_bridge

**Tu tablero de tareas y tu fuente única de verdad,
impecablemente sincronizados — en ambos sentidos.**

Resumen

task_bridge es un motor genérico, desacoplado y bidireccional de sincronización de tareas/tableros en Go. Mantiene sincronizada la fuente única de verdad de los elementos procesables SQLite de un proyecto con sus documentos de seguimiento y un tablero remoto (objetivo inicial: ClickUp; previstos Jira/Linear), mediante una semántica determinista de *última edición gana, ejecución en seco primero y nunca corrompe*.

Descripción breve

Submódulo de Go agnóstico a proyectos que sincroniza de forma bidireccional una fuente única de verdad de elementos procesables SQLite ↔ documentos de seguimiento ↔ tablero remoto (ClickUp primero). *Última edición gana* determinista, *ejecución en seco primero*, webhooks verificados con HMAC; todas las credenciales e IDs son inyectadas por el consumidor en tiempo de ejecución.

Descripción detallada

Tarde o temprano, todo equipo gestiona dos registros del mismo trabajo: el real —código, documentación, una base de datos interna— y el que vigilan los responsables, un tablero como ClickUp. Ambos se desincronizan en cuanto se toca cualquiera de los dos, y reconciliarlos a mano es justo el tipo de tarea tediosa y propensa a errores que nadie realiza con fiabilidad. task_bridge se creó para eliminar esa brecha, tratando las tres representaciones como un único sistema que debe mantenerse en perfecta sincronía: la **fuentes única de verdad de los elementos procesables SQLite** de un proyecto, su **documentación de seguimiento** y un **tablero remoto** —el primero compatible es

ClickUp, con Jira y Linear previstos para futuras versiones—. La sincronización es determinista (*última edición gana*), se ejecuta primero en modo prueba y está diseñada en torno a una promesa innegociable: nunca corromperá ni perderá datos, ni dejará silenciosamente desactualizado uno de los lados. En un ámbito donde un error en la sincronización puede borrar una semana de trabajo, esa garantía de seguridad es el objetivo principal.

Desde el punto de vista arquitectónico, es un submódulo estricto consumido por otros proyectos y completamente agnóstico a ellos, según el contrato de desacoplamiento de la constitución (§11.4.28): no incluye valores específicos de ningún proyecto, y todas las credenciales, IDs de tableros/carpetas, campos de claves de elementos y rutas de bases de datos son inyectados por el consumidor en tiempo de ejecución a través de `pkg/config.Config`. El módulo está estructurado en capas bien definidas: un CLI (`reconcile/push/pull/resolve/status/conflicts/init`) y un demonio de ejecución prolongada (receptor de webhooks + cron de reconciliación); un cliente ligero sobre `raksul/go-clickup` (licencia MIT); un resolutor que convierte URLs de tableros/carpetas en IDs mediante sondeos en vivo de API (sin adivinar gramáticas de URL); un mapeador entre elementos procesables locales y campos de tareas remotas; un motor de sincronización *última edición gana* con resultados explícitos de conflictos; y un receptor de webhooks que verifica X-Signature con HMAC-SHA256. Es transparente sobre su estado de madurez: esta es la estructura prioritaria (P1) —el diseño, las interfaces, los puntos de entrada y el límite de desacoplamiento están definidos—, pero la lógica de sincronización y las llamadas en vivo a ClickUp aún no están implementadas (cada *stub* devuelve un error explícito de *no implementado*, según la regla de *sin falsificaciones*).

Por qué lo creamos

Los equipos mantienen el “estado real” del trabajo en código y documentación, mientras que los responsables viven en un tablero como ClickUp —y ambos se desvían constantemente—. `task_bridge` los convierte en un único sistema, sincronizándolos de forma determinista y segura para que ninguno de los dos lados quede obsoleto o incorrecto.

Por qué es un cambio de juego

La sincronización bidireccional de tableros suele ser una integración única y rígida que cada equipo reconstruye de manera deficiente. `task_bridge` la redefine como una biblioteca reutilizable con inyección de credenciales y garantías estrictas de seguridad de datos integradas: primero simulación, determinismo en la última edición prevaleciente, eventos verificados con HMAC. Así,

cualquier proyecto puede adoptar una integración de tableros confiable mediante la inyección de configuración, en lugar de escribir otro conector frágil acoplado a sus componentes internos.

Qué tiene de innovador

- Sincronización bidireccional en tres vías: SQLite (SSoT) ↔ documentos del rastreador ↔ tablero remoto.
- Desacoplamiento total (§11.4.28): sin valores de proyecto; todo se inyecta en tiempo de ejecución.
- Resolución en vivo API → ID en lugar de análisis sintáctico frágil de gramáticas URL.
- Ingestión de webhooks con verificación HMAC-SHA256 para eventos en tiempo real.

Desafíos y soluciones

- **Seguridad de datos entre tres fuentes:** resuelto con determinismo en la última edición prevaleciente, simulación previa y resultados explícitos de conflictos.
- **Reutilización sin acoplamiento:** resuelto mediante el límite de inyección pkg/config (sin detalles específicos del proyecto).
- **Identificación confiable de tableros:** resuelto al convertir URLs en IDs mediante sondeos en vivo con API.
- **Andamiaje honesto:** resuelto al hacer que los *stubs* no implementados devuelvan errores explícitos de “no implementado” (sin falsificaciones).

Tecnologías empleadas (por qué y cómo)

- **Go** — motor, CLI (cmd/task_bridge) y demonio (cmd/task_bridged).
- **SQLite** — la fuente única de verdad para elementos procesables.
- **raksul/go-clickup (MIT)** — envoltorio de transporte para ClickUp.
- **HMAC-SHA256** — verificación de firmas de webhooks.
- **cron + webhooks** — reconciliación del demonio e ingestión de eventos en vivo.
- **pkg/config** — límite de inyección de credenciales/IDs en tiempo de ejecución.

Honestidad de estado: esto es un **andamiaje P1** — la lógica de sincronización aún no está implementada. No debe presentarse como producto finalizado.